

GUIA 04: Análisis, Arquitectura y diseño de software. Documentación C4 en un sistema Marketplace

ASIGNATURA	IS-488 ARQUITECTURA DE SOFTWARE
DOCENTE	ING. LIZBETH JAICO QUISPE
SEMESTRE	2026-II
LABORATORIO	04, 05,06, 07
SEMANA	4-5-6
CASO DE ESTUDIO	Marketplace de productos para mascotas: GoPet: https://www.gopet.pe/
DURACIÓN	2 horas

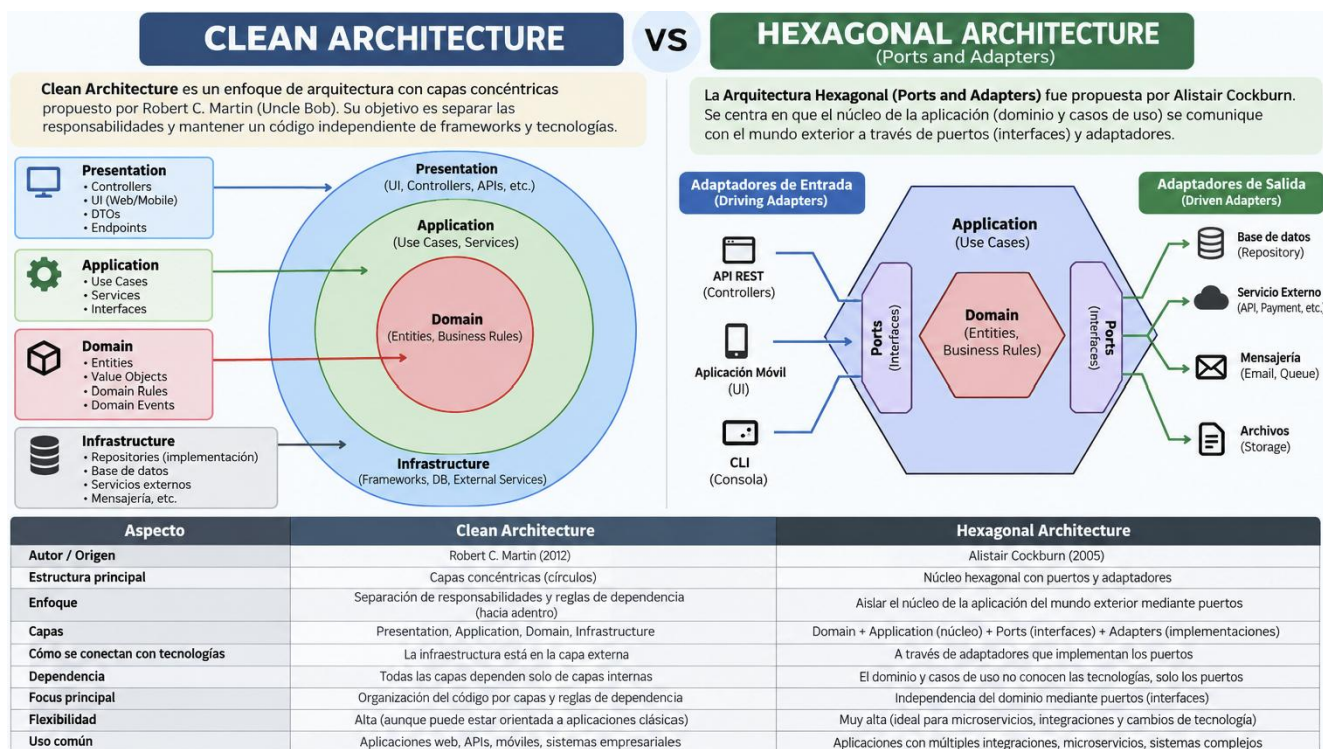
I. PROPÓSITO DEL LABORATORIO

En la Guía 03 se realizó lo más exigente del curso: pasar de entender una arquitectura a producir la documentación que la sustenta, de modo que otra persona pueda construir el sistema a partir de ella.

El trabajo abarca cuatro sesiones de laboratorio y culmina en la presentación del primer sprint. Al terminar este sprint, cada estudiante o equipo tendrá su propio repositorio de su proyecto con la documentación completa: análisis del sistema, arquitectura de software y diseño de software, organizada de manera que permita comprender, justificar y comunicar la solución propuesta.

III. MARCO CONCEPTUAL

1. ENFOQUE: Clean Architecture vs Hexagonal Architecture.



Criterio	Clean Architecture (Robert C. Martin)	Hexagonal Architecture (Ports and Adapters - Alistair Cockburn)
Objetivo principal	Mantener un código independiente de frameworks y tecnologías , separando claramente las responsabilidades en capas (Presentación, Aplicación, Dominio, Infraestructura).	Aislar el núcleo de la aplicación del mundo exterior mediante puertos (interfaces) y adaptadores (implementaciones), facilitando el cambio de tecnologías externas sin afectar la lógica de negocio.
Enfoque	Enfoque general de arquitectura y diseño de software basado en principios y capas .	Enfoque específico que implementa la separación del núcleo usando puertos y adaptadores (es una forma de aplicar Clean Architecture).
Estructura	Capas concéntricas: - Presentación (UI, Controllers, APIs) - Aplicación (Use Cases, Services) - Dominio (Entities, Business Rules) - Infraestructura (DB, External Services)	Núcleo con puertos y adaptadores: - Dominio + Casos de uso (núcleo) - Puertos (interfaces de entrada y salida) - Adaptadores (implementaciones para UI, BD, APIs, etc.)
Cuándo usar	- Proyectos de mediana a gran escala. - Lógica de negocio compleja y crítica. - Equipos de desarrollo medianos o grandes. - Necesitas alta mantenibilidad, escalabilidad y pruebas. - Quieres una arquitectura clara y organizada por capas.	- Proyectos con múltiples integraciones externas. - Microservicios o sistemas distribuidos. - Aplicaciones con cambios frecuentes de tecnología. - Necesitas aislar el dominio del mundo exterior. - Proyectos event-driven, APIs, mensajería, o múltiples canales (web, móvil, CLI, etc.).
Cuándo NO usar	- Proyectos muy pequeños o simples. - Cuando necesitas entregar muy rápido (MVP muy básico). - Equipos muy pequeños sin experiencia en arquitectura.	- Proyectos simples o con pocas integraciones. - Cuando no necesitas aislar el dominio de tecnologías externas. - Si el proyecto es temporal o desechable.
Ventajas	- Código independiente de frameworks. - Alta mantenibilidad y testabilidad. - Separación de responsabilidades clara. - Escalable a largo plazo.	- Fácil cambio de tecnologías externas. - Facilita pruebas (puertos permiten mocks). - Ideal para múltiples integraciones. - Gran flexibilidad para entornos distribuidos.
Desventajas	- Puede ser más complejo de entender al inicio. - Más estructura (puede ser sobreingeniería en proyectos pequeños). - Mayor tiempo inicial de diseño.	- Requiere disciplina en el diseño de puertos y adaptadores. - Puede ser más complejo en proyectos muy simples. - Necesita un buen entendimiento del dominio y las interfaces.
Ejemplos de proyectos	Ejemplos donde usar Clean Architecture E-commerce completo Plataforma de ventas con lógica de negocio compleja (productos, pagos, envíos, descuentos). Sistema bancario Gestión de cuentas, transferencias, seguridad y reglas de negocio críticas. Sistema de salud Gestión de pacientes, historias clínicas, procesos complejos y normativas. ERP empresarial Múltiples módulos (ventas, inventario, RRHH, finanzas).	Ejemplos donde usar Hexagonal Architecture Microservicio de pagos Integración con pasarelas de pago (ej. Culqi, Nuobiz, Stripe), fácil cambio de proveedor. Servicio de notificaciones Integración con email, SMS, WhatsApp, push (que pueden cambiar de proveedor). API Gateway Conexión a múltiples servicios externos y adaptadores para diferentes clientes. Sistema IoT / Dispositivos Integración con sensores, dispositivos y múltiples protocolos (MQTT, HTTP, WebSocket).

2. Escenarios Atributos de Calidad

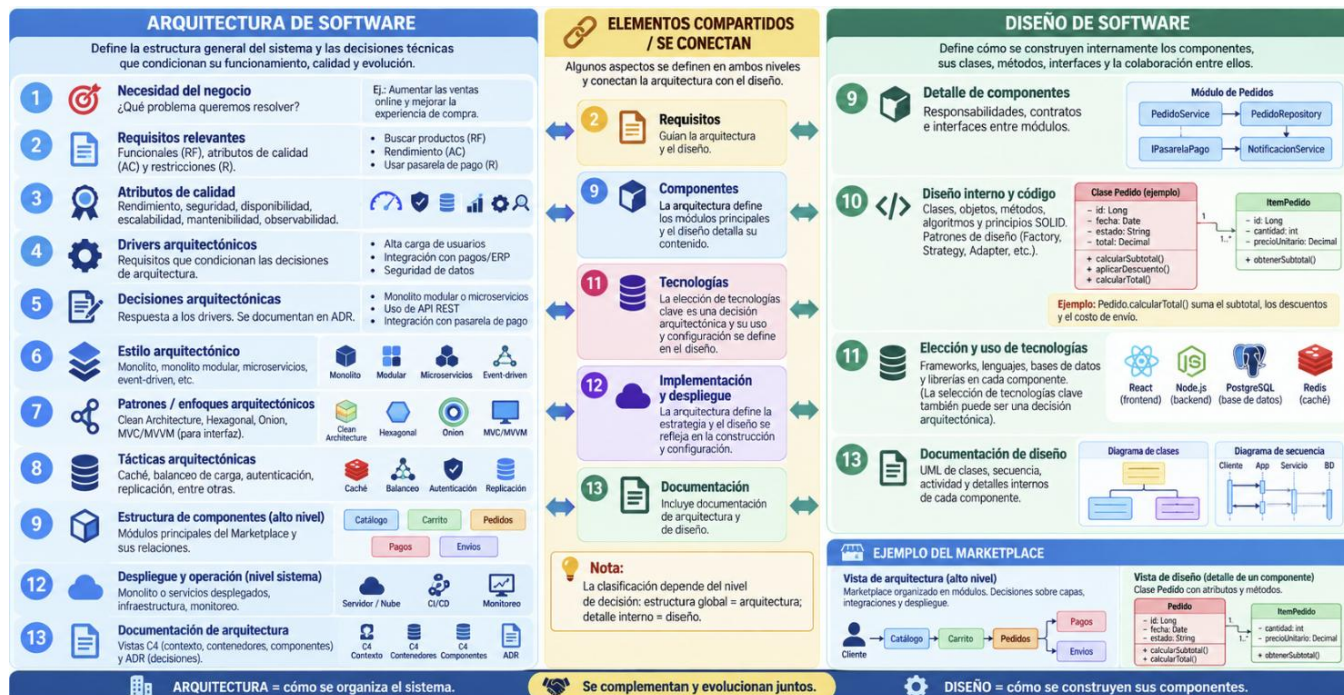
Es una descripción concreta y medible de cómo el sistema debe responder ante un estímulo para cumplir un atributo de calidad. Su objetivo es definir y validar los requisitos de calidad del sistema de forma clara, medible y verificable para guiar el diseño arquitectónico y las decisiones técnicas



3. Diferencia entre Arquitectura de Software y Diseño de Software

- La **arquitectura de software** define la estructura general del sistema, sus componentes principales, cómo se relacionan y las decisiones técnicas que condicionan su funcionamiento y evolución.

- El **diseño de software** define cómo se construyen internamente esos componentes, especificando sus clases, métodos, responsabilidades, interfaces y la colaboración entre ellos para implementar las funcionalidades del sistema.



Ambos se complementan: la arquitectura establece la organización general y sus principales reglas; el diseño detalla cómo implementar cada parte respetando esas decisiones.

Patrones de Diseño. Los patrones de diseño son soluciones probadas y reutilizables que ayudan a los desarrolladores a resolver problemas comunes al diseñar software. Permiten estandarizar la forma de organizar el código, facilitar el trabajo en equipo, reducir el tiempo de desarrollo y hacer que el software sea más fácil de mantener, modificar y evolucionar.



Patrones de Diseño

Son soluciones probadas y reutilizables que ayudan a los desarrolladores a resolver problemas comunes al diseñar software. Permiten estandarizar la forma de organizar el código, facilitar el trabajo en equipo, reducir el tiempo de desarrollo y hacer que el software sea más fácil de mantener, modificar y evolucionar.



En pocas palabras...

Un patrón de diseño es como una "receta" para resolver un problema frecuente. No es código listo para copiar, sino una guía de cómo organizar la solución.

01 Factory (Fábrica)

¿Quién crea los objetos?

Problema

Crear objetos directamente en el código puede ser inflexible y difícil de mantener, sobre todo cuando hay muchos tipos.

Solución

Usamos una fábrica que se encarga de crear los objetos según lo que necesitamos.

Ejemplo en e-commerce:
Crear diferentes tipos de productos.



```
const producto = factory.crearProducto('laptop');
// La fábrica se encarga de la creación
```

02 Facade (Fachada)

¿Hay demasiadas cosas que llamar?

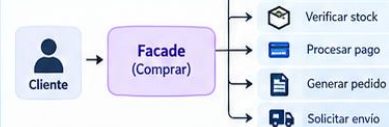
Problema

El cliente debe conocer y llamar a varios servicios o componentes para completar una acción.

Solución

Creamos una interfaz simple que encapsule toda la lógica compleja.

Ejemplo en e-commerce:
Comprar un producto.



El cliente solo llama a: comprar()

03 Chain of Responsibility (Cadena de Responsabilidad)

¿Quién debe atender la solicitud?

Problema

Una solicitud puede requerir varias validaciones o procesos en diferentes niveles.

Solución

La solicitud pasa por una cadena de manejadores, hasta que uno de ellos la atiende.

Ejemplo en e-commerce:
Validación de una compra.



```
if (usuarioValido) {
  if (stockDisponible) {
    if (pagoAprobado) { ... }
  }
}
```

04 Circuit Breaker

¿Qué hacemos si un servicio externo falla?

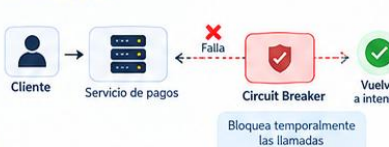
Problema

Si un servicio externo falla, la aplicación puede seguir intentando llamarlo y causar más errores.

Solución

El patrón "circuit breaker" detecta fallos y bloquea llamadas temporales para evitar más problemas.

Ejemplo en e-commerce:
Servicio de pagos.



05 Adapter (Adaptador)

¿Qué hacemos cuando las interfaces no son compatibles?

Problema

A veces usamos servicios o librerías externas que tienen una interfaz diferente a la que necesitamos.

Solución

Creamos un adaptador que traduce la interfaz externa a la que nuestro sistema usa.

Ejemplo en e-commerce:
Integrar un proveedor de pagos externo.



```
class PagoAdapter {
  procesarPago(datos) {
    return proveedor.pagar(formatear(datos));
  }
}
```

06 CORS (Cross-Origin Resource Sharing)

¿Quién puede acceder a mi API?

Problema

Por seguridad, el navegador bloquea peticiones desde otros dominios (orígenes) a tu API.

Solución

Se configura CORS para permitir solo los orígenes autorizados.

Ejemplo en e-commerce:
Permitir que el frontend consuma el backend.



```
res.setHeader('Access-Control-Allow-Origin', 'https://mi-tienda.com');
```



En resumen

✓ Resuelven problemas comunes de desarrollo.

✓ Hacen el código más claro y organizado.

✓ Facilitan el trabajo en equipo.

✓ Ahorran tiempo y esfuerzo.

✓ Mejoran la calidad y mantenibilidad del software.

No se trata de memorizar patrones, sino de entender el problema, la solución y cuándo aplicarlo.

Diseño de software:

Principio de Diseño- SOLID. son un conjunto de cinco reglas básicas para el diseño y la programación orientada a objetos (POO). Su objetivo es ayudar a los desarrolladores a escribir código más limpio, fácil de mantener, flexible y escalable



Principios SOLID en un e-commerce peruano

Métodos de pago: Yape, Plin y Tarjeta

Yape, Plin y tarjeta son ejemplos de implementaciones del sistema.
Los principios SOLID se aplican al código que las integra.



Yape
QR / Internet



Plin
Número de celular / QR



Tarjeta
Compras por internet

1

SRP - Responsabilidad Única

Una clase = una razón principal para cambiar

❌ Problema: una clase hace todo

Si una clase realiza muchas tareas, cualquier cambio afecta todo el sistema.

```
class Checkout {  
    // valida pedido  
    // calcula total  
    // procesa pago Yape/Plin/Tarjeta  
    // envía correo  
    // guarda en base de datos  
}
```

Difícil de mantener y con muchos cambios cuando hay nuevas reglas.

✅ Solución: una responsabilidad por clase

**2**

OCP - Abierto/Cerrado

Agrego nuevas formas de pago sin tocar el código existente

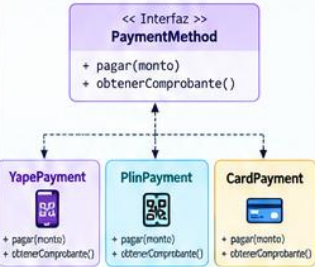
❌ Problema: cada nuevo método obliga a modificar el código

Si aparece un nuevo método de pago, debo cambiar la clase Checkout.

```
class Checkout {  
    public void pagar(String tipo) {  
        if (tipo.equals("yape")) {  
            // procesa Yape  
        } else if (tipo.equals("plin")) {  
            // procesa Plin  
        } else if (tipo.equals("tarjeta")) {  
            // procesa Tarjeta  
        }  
    }  
}
```

Cada nuevo método de pago implica modificar esta clase.

✅ Solución: interfaz y nuevas implementaciones



Para agregar un nuevo método de pago se crea una nueva implementación sin modificar Checkout.

3

LSP - Sustitución de Liskov

Si prometen el mismo contrato, deben poder reemplazarse sin romper el sistema

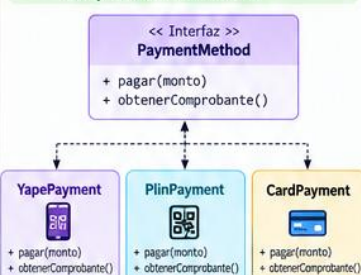
❌ Problema: implementaciones que no cumplen el contrato

Si una implementación no se comporta como se espera, el sistema puede fallar.

```
class YapePayment {  
    // método con nombre diferente  
    // o no implementa pagar()  
    public void realizarPago() { }  
}
```

Si no cumple el mismo contrato, no se puede reemplazar.

✅ Solución: todas las implementaciones cumplen la misma interfaz



Cualquier implementación puede reemplazar a otra y Checkout seguirá funcionando.

4

ISP - Segregación de Interfaces

No obligues a usar métodos que no necesitas

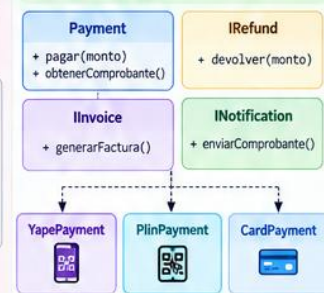
❌ Problema: interfaz muy grande

Un cliente que solo necesita pagar depende de muchos métodos que no usa.

```
interface PaymentService {  
    pagar(monto);  
    devolver(monto);  
    generarFactura();  
    enviarComprobante();  
    validarIdentidad();  
}
```

Obliga a implementar métodos innecesarios.

✅ Solución: interfaces pequeñas y específicas



Cada clase implementa solo las interfaces que necesita.

5

DIP - Inversión de Dependencias

Depende de contratos, no de implementaciones concretas

❌ Problema: depende de una implementación concreta

Si Checkout crea directamente un método de pago, queda acoplado a esa implementación.

```
class Checkout {  
    public void pagar() {  
        YapePayment yape = new YapePayment(); // dependencia directa  
        yape.pagar(monto);  
    }  
}
```

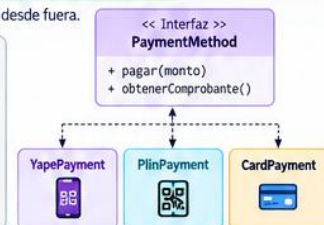
Si quiero cambiar a Plin o a otro proveedor, debo modificar Checkout.

✅ Solución: depende de una abstracción (inyección de dependencias)

Checkout recibe una implementación de PaymentMethod desde fuera.

```
class Checkout {  
    private PaymentMethod paymentMethod;  
    public Checkout(PaymentMethod paymentMethod) {  
        this.paymentMethod = paymentMethod;  
    }  
    public void pagar() {  
        paymentMethod.pagar(monto);  
    }  
}
```

Checkout funciona con cualquier implementación (Yape, Plin, Tarjeta) sin modificar su código.



SOLID en una sola idea

No se trata de usar Yape, Plin o tarjetas; se trata de diseñar el código para que agregar, cambiar o mantener métodos de pago sea sencillo y seguro.

Analogía: una tienda



SOLID es como organizar una tienda: cada empleado tiene una función, puedes agregar nuevos servicios sin rehacer la tienda y cada parte depende de reglas claras.

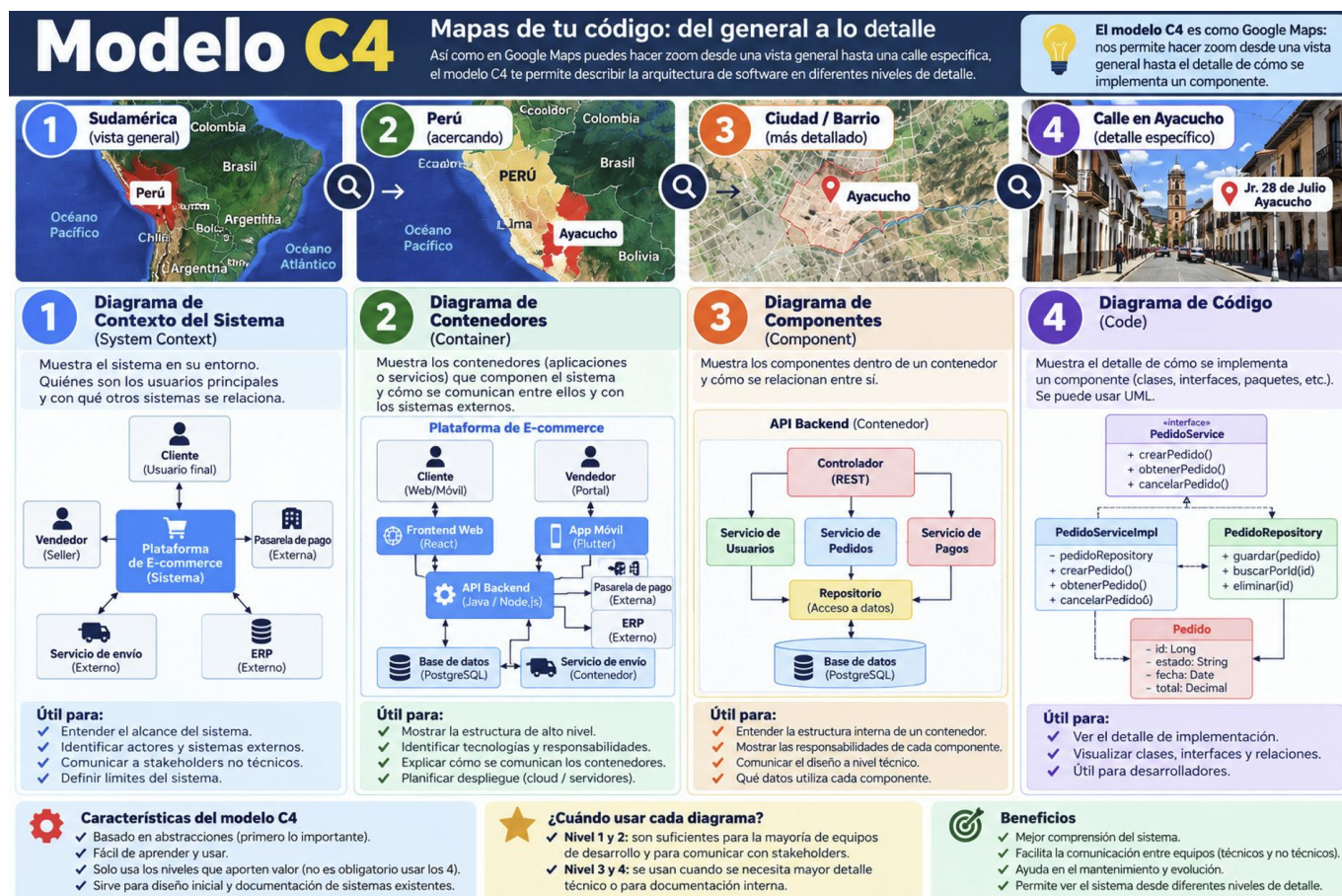
4. Documentación modelo C4.

El modelo C4 es una forma de representar y documentar la arquitectura de software mediante diagramas organizados en cuatro niveles de detalle: contexto, contenedores, componentes y código.

Permite comprender el sistema desde una visión general hasta su estructura interna, mostrando sus usuarios, sus relaciones con otros sistemas, la organización de sus aplicaciones y los componentes que las conforman.

Su objetivo es facilitar la comunicación de la arquitectura entre desarrolladores, arquitectos, responsables del proyecto y otras personas involucradas.

Nivel	Qué se documenta	Ejemplo del Marketplace
C1. Contexto	Usuarios, sistema principal y sistemas externos.	Cliente, vendedor, Marketplace, pasarela de pago y servicio de envío.
C2. Contenedores	Aplicaciones, servicios ejecutables y almacenes de datos.	Aplicación web, API, PostgreSQL y caché.
C3. Componentes	Responsabilidades y relaciones dentro de una aplicación o contenedor.	Catálogo, carrito, pedidos y pagos.
C4. Código	Estructura interna de un componente.	Clases, interfaces y métodos para calcular el total de un pedido.



IV. CONTEXTO DEL CASO DE ESTUDIO

Se desea desarrollar un marketplace de productos para mascotas, en el cual diferentes sellers puedan ofrecer sus productos y los clientes puedan realizar compras.

Para comprender el contexto del negocio se tomará como referencia funcional un marketplace como GoPet.

Los principales participantes del sistema son: cliente, seller, administrador, Pasarela de pagos, servicio de envíos, ERP

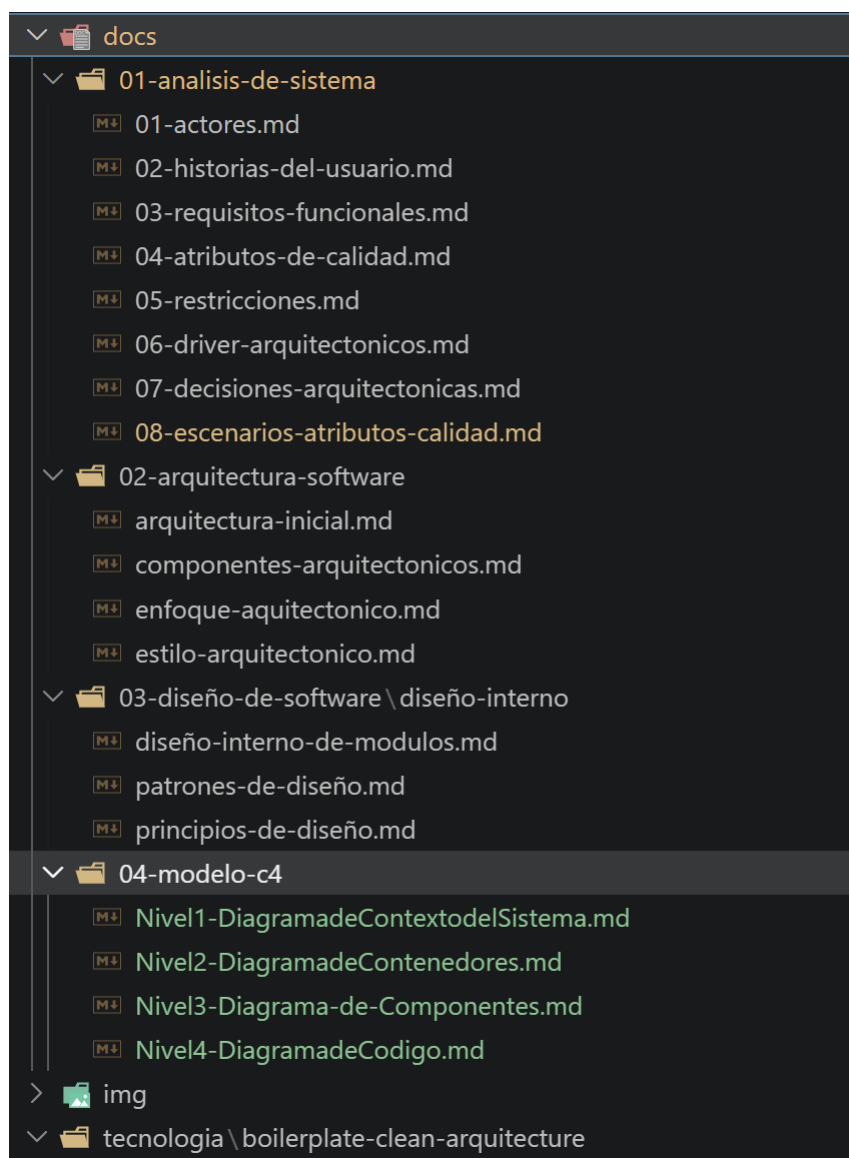
V. REPASO Y MODIFICACIÓN DE LA GUIA 2

PASO 01: REESTRUCTURAR LAS CARPETAS DEL PROYECTO

En la Guía 03 se definió trabajar con una arquitectura monolítica modular organizada en capas. Después de desarrollar el análisis del sistema y el diseño arquitectónico, se organizará la documentación del proyecto.

Procedimiento:

1. Abrir el proyecto Marketplace-arquitSoft-02 en Visual Studio Code.
2. Organizar los archivos en dos directorios principales: docs y tecnologia.
3. Guardar en docs los documentos de análisis, arquitectura, diseño de software, diagramas C4 e imágenes.
4. Guardar en tecnologia el ejemplos de código del enfoque clean architecture.
5. Verificar que los archivos estén correctamente ubicados y que las rutas de las imágenes funcionen.



Resultado esperado: un proyecto ordenado, con la documentación separada de los recursos tecnológicos. Posterior a ello realizar su primer commit.

PASO 2: Convertir los atributos de calidad en escenarios medibles

En la Guía 02 cada atributo de calidad quedó enunciado como una descripción general: «el sistema debe responder rápidamente», «el sistema debe estar disponible». Una descripción no es un escenario. No dice quién origina la exigencia, bajo qué condiciones ocurre ni con qué valor se comprueba si se cumplió, de modo que no puede evaluarse.

Un escenario de atributo de calidad se escribe con seis partes. Completa esta estructura para cada uno de tus atributos, en el archivo docs/01-analisis-de-sistema/08-escenarios-atributos-calidad.md.

Parte	Qué responde	Ejemplo en el Marketplace
Fuente del estímulo	¿Quién o qué lo origina?	Clientes que navegan el catálogo durante una campaña
Estímulo	¿Qué ocurre?	Peticiones simultáneas de búsqueda de productos
Artefacto	¿Sobre qué parte del sistema?	Módulo Catálogo y su acceso a la base de datos
Entorno	¿En qué condiciones?	Campaña comercial, con la concurrencia esperada
Respuesta	¿Qué debe hacer el sistema?	Devolver los resultados sin rechazar peticiones
Medida de la respuesta	¿Con qué valor se comprueba?	Percentil 95 por debajo del tiempo objetivo; ningún error 5xx

Archivo: 08-escenarios-atributos-calidad.md

```
# Escenarios de atributos de calidad del Marketplace

## EQ-01. Rendimiento: consulta de productos

| Elemento | Descripción |
| --- | --- |
| **Atributo de calidad** | Rendimiento |
| **Fuente** | Cliente |
| **Estímulo** | El cliente busca productos o consulta el detalle de un producto. |
| **Condición** | El Marketplace funciona normalmente y se utilizan datos representativos. |
| **Respuesta esperada** | El sistema muestra los resultados de búsqueda o los detalles del producto sin errores inesperados. |
| **Medida** | **Propuesta por validar:** el 95 % de las consultas responde en 2 segundos o menos, con menos del 1 % de errores técnicos, durante una prueba con 200 usuarios simultáneos. |
| **Verificación** | Ejecutar pruebas de rendimiento y registrar los tiempos de respuesta, los errores y la cantidad de usuarios simultáneos. |
| **Estado** | Pendiente de aprobación de las medidas y definición del entorno de prueba. |

## EQ-02. Disponibilidad: continuidad del servicio

| Elemento | Descripción |
| --- | --- |
| **Atributo de calidad** | Disponibilidad |
| **Fuente** | Cliente |
| **Estímulo** | El cliente intenta realizar una operación durante una campaña comercial o cuando ocurre una falla en un componente. |
| **Condición** | El sistema está publicado y cuenta con monitoreo. |
| **Respuesta esperada** | El sistema mantiene disponibles las operaciones que puede atender y comunica los errores de forma controlada cuando un servicio necesario no responde. |
| **Medida** | **Propuesta por validar:** disponibilidad mensual de 99,5 % para las operaciones críticas. |
```

| ****Verificación**** | Revisar los registros de monitoreo y calcular los periodos de disponibilidad e interrupción. |

| ****Estado**** | Pendiente de aprobar la meta, definir las operaciones críticas y establecer las condiciones de mantenimiento. |

EQ-03. Escalabilidad: aumento de usuarios

| Elemento | Descripción |

---|---

| ****Atributo de calidad**** | Escalabilidad |

| ****Fuente**** | Incremento de solicitudes de los clientes |

| ****Estímulo**** | Aumenta la cantidad de usuarios que utilizan el Marketplace simultáneamente. |

| ****Condición**** | Se realiza una prueba con datos, configuración y solicitudes documentados. |

| ****Respuesta esperada**** | El sistema soporta el aumento de usuarios sin superar los límites acordados de tiempo de respuesta y errores. |

| ****Medida**** | ****Propuesta por validar:**** soportar hasta 200 usuarios simultáneos, con el 95 % de las respuestas por debajo de 3 segundos y menos del 1 % de errores técnicos. |

| ****Verificación**** | Aumentar progresivamente la carga y comparar los tiempos de respuesta, los errores y el uso de recursos. |

| ****Estado**** | Pendiente de confirmar la cantidad esperada de usuarios y la infraestructura de prueba. |

EQ-04. Seguridad: acceso a operaciones protegidas

| Elemento | Descripción |

---|---

| ****Atributo de calidad**** | Seguridad |

| ****Fuente**** | Usuario sin sesión, con credenciales inválidas o sin permisos suficientes |

| ****Estímulo**** | El usuario intenta acceder a una función protegida o realizar una operación no autorizada. |

| ****Condición**** | Se realizan pruebas sobre las funciones protegidas del Marketplace. |

| ****Respuesta esperada**** | El sistema rechaza la operación, no ejecuta el cambio solicitado y no expone información sensible. |

| ****Medida**** | Rechazar el 100 % de los intentos no autorizados incluidos en las pruebas definidas, sin exposición de datos sensibles en las respuestas revisadas. |

| ****Verificación**** | Ejecutar pruebas de acceso autorizado y no autorizado, revisar las respuestas y comprobar que no se expongan datos sensibles. |

| ****Estado**** | Pendiente de definir los permisos por tipo de usuario y las operaciones protegidas. |

EQ-05. Mantenibilidad: cambios sin afectar otras funciones

| Elemento | Descripción |

---|---

| ****Atributo de calidad**** | Mantenibilidad |

| ****Fuente**** | Equipo de desarrollo |

| ****Estímulo**** | Se modifica una regla de negocio o se necesita sustituir el proveedor de pagos. |

| ****Condición**** | El sistema está organizado en módulos y dispone de pruebas para las funciones afectadas. |

| ****Respuesta esperada**** | El cambio se concentra en el módulo responsable sin afectar funciones no relacionadas, salvo que exista una dependencia justificada. |

```
| **Medida** | Las pruebas del módulo modificado y las pruebas de regresión acordadas deben
aprobarse. Las dependencias entre módulos deben respetar las reglas de arquitectura establecidas. |
| **Verificación** | Revisar los cambios de código, ejecutar las pruebas e inspeccionar las
dependencias entre módulos. |
| **Estado** | Pendiente de definir las reglas de dependencias y las pruebas obligatorias para los
cambios. |
```

Resumen de escenarios

```
| Código | Atributo de calidad | Qué se busca comprobar |
|---|---|---|
| EQ-01 | Rendimiento | Que el Marketplace responda dentro del tiempo establecido. |
| EQ-02 | Disponibilidad | Que las operaciones críticas estén disponibles cuando se necesiten. |
| EQ-03 | Escalabilidad | Que el sistema soporte un aumento de usuarios dentro de los límites
definidos. |
| EQ-04 | Seguridad | Que las operaciones protegidas no puedan ejecutarse sin autorización. |
| EQ-05 | Mantenibilidad | Que los cambios puedan realizarse sin afectar innecesariamente otras
funciones. |
```

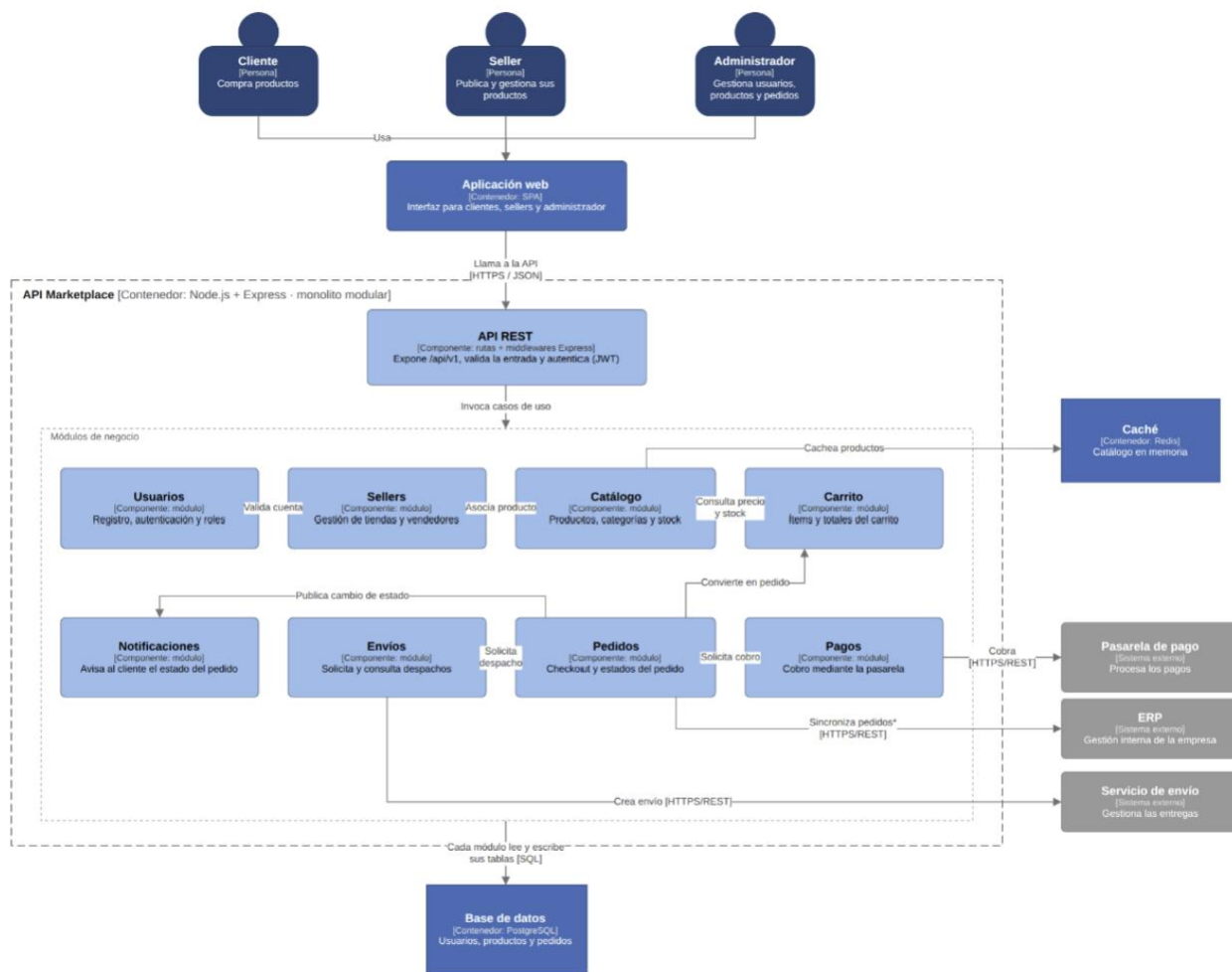
PASO 03: Componentes arquitectónicos.

Agregar la imagen en el archivo md.

[Diagrama de componentes] Marketplace – API Backend

Modelo C4 - Nivel 3 (componentes) - Estilo: monolito modular en capas

Vista de componentes - v



Notas

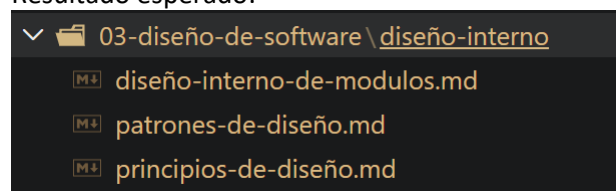
- Cada componente es un módulo del monolito: todos se despliegan juntos.
- Un módulo usa a otro solo a través de su interfaz pública (servicio), nunca sus tablas.
- * La integración con el ERP se asigna a Pedidos: confirmar con el negocio.

PASO 04: DISEÑO DE SOFTWARE

Agregar estos archivos.

<https://github.com/devlizbethjaico/Marketplace-arquitSoft-02/tree/main/docs/03-dise%C3%B1o-de-software/dise%C3%B1o-interno>

Resultado esperado:



PASO 05: MODELO C4.

<https://github.com/devlizbethjaico/Marketplace-arquitSoft-02/tree/main/docs/04-modelo-c4>

VII. TAREA

Para la siguiente sesión, Entregable del primer sprint del proyecto final. Se deberá presentar de su proyecto toda la documentación de su proyecto. (el entregable será mediante exposición).

- El entregable del primer sprint es la documentación completa del proyecto propio, versionada en el repositorio del equipo, y su exposición en clase. Se entrega la URL del repositorio, no archivos sueltos.
- **La exposición:** 12 minutos por equipo. Expone todo el equipo, y cualquier integrante debe poder responder de qué requisito sale cualquier componente del diagrama). En base a esta exposición se también se calificará los entregables anteriores)